Database Schema Document — Library Management System

1. Document purpose

This document defines the database schema for the Library Management System built with FastAPI and PostgreSQL. It converts the agreed business rules into a clean, practical relational design suitable for a small library of about 200 books while remaining scalable for future enhancements.[1][2]

The design follows the simplified model requested for this project: a single users table for authentication, a books table at title level, an issued_books table for circulation, a waitlist table for queue handling, and a book_requests table for acquisition requests. It intentionally avoids a separate physical-copy table in the current version to keep implementation fast and manageable for the project scope.[3][4]

2. Design goals

- Keep the schema practical and easy to implement for a college-scale library system.[3][4]
- Support two roles, librarian and student, through a single authentication table.[5][3]
- Enforce the core lending rules: 7-day issue period, ₹5/day fine, duplicate-issue prevention, waitlist ordering, and overdue tracking.[6][7]
- Use PostgreSQL constraints, indexes, and transaction-safe operations to preserve data integrity during issue and return flows.[8][1]

3. Scope of schema

The schema covers authentication, catalog metadata, issue/return tracking, waitlist handling, and book requests. It is optimized for title-level inventory counts rather than copy-level physical tracking, which is a deliberate tradeoff for simplicity in this version.[3][6][4]

This means the system tracks `total_copies` and `available_copies` in the `books` table instead of modeling each individual copy. That approach is acceptable for the current scale, but it also means the application cannot identify which exact physical copy was issued, damaged, or lost unless a later version introduces a `book_copies` table.[6][4]

4. Core design decisions

4.1 Single users table

A single `users` table stores both librarians and students, with a role column controlling access behavior in the backend. This simplifies authentication, avoids duplicated credential structures, and supports RBAC cleanly at the API layer.[5][3][9]

4.2 No separate copies table

This version does not create a separate copies table. Instead, inventory is maintained using aggregate counts on the `books` table because the project currently prioritizes practical implementation over detailed physical-copy tracking.[3][4]

4.3 Transaction-safe inventory updates

Issue and return flows must update both circulation records and book availability counters inside a single transaction. This is necessary so the system does not create inconsistent states such as an issued record without stock reduction or a returned record without stock restoration.[10][1][11]

4.4 Waitlist as a queue

The `waitlist` table models queue order per book using a position or timestamp-based approach. The backend must always allocate returned stock to the earliest eligible student before general availability is increased for open circulation.[6][7]

5. Entity overview

Table	Purpose	Main relationship
`users`	Stores librarian and student accounts	One user can have many issued books, waitlist entries, and book requests.[3]
`books`	Stores title-level metadata and inventory counts	One book can have many issue records, waitlist entries, and requests indirectly.[6][4]
`issued_books`	Stores loan transactions	Many issue records belong to one user and one book.[6][4]
`waitlist`	Stores queue entries for unavailable books	Many waitlist entries belong to one user and one book.[6][7]
`book_requests`	Stores student requests for new books	Many requests belong to one user.[6]

6. Table definitions

6.1 Users table

```sql
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('librarian', 'student')),
```

```
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Purpose

This table stores login credentials and identity data for both librarians and students. The `role` field is the basis for backend role-based access control.[5][3]

Notes

- `email` must be unique because it acts as the primary login identifier.[3]
- `password_hash` stores only hashed passwords, never plain text.[5][3]
- `role` is restricted through a check constraint to valid system roles.[2][12]

6.2 Books table

```
CREATE TABLE books (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    isbn VARCHAR(20) UNIQUE,
    total_copies INT NOT NULL CHECK (total_copies >= 0),
    available_copies INT NOT NULL CHECK (available_copies >= 0),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_available_le_total CHECK (available_copies <= total_co
);
```

Purpose

This table stores book metadata and title-level inventory counts. It does not represent physical copies individually.[6][4]

Notes

- total_copies is the library's count of all owned copies of the title.[4]
- available_copies represents how many copies are currently free for issue.[6]
- The additional check available_copies <= total_copies is important to stop invalid stock states.[2][12]
- isbn is unique when present, but should be nullable because some books may not have a reliable ISBN in smaller institutional libraries.[4]

6.3 Issued books table

```
CREATE TABLE issued_books (
  id SERIAL PRIMARY KEY,
  user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  book_id INT NOT NULL REFERENCES books(id) ON DELETE CASCADE,
  issue_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  due_date TIMESTAMP NOT NULL,
  return_date TIMESTAMP,
  fine_amount INT NOT NULL DEFAULT 0 CHECK (fine_amount >= 0),
  status VARCHAR(20) NOT NULL CHECK (status IN ('issued', 'returned', 'ov
  CONSTRAINT chk_due_after_issue CHECK (due_date >= issue_date),
  CONSTRAINT chk_return_after_issue CHECK (return_date IS NULL OR re
);
```

Purpose

This table stores each issue transaction and its return status. It is the core circulation table for due dates, overdue detection, and fine tracking.[6][13][4]

Notes

- due_date should be assigned by backend logic as issue_date + interval '7 days'.[6][13]
- fine_amount is stored as a final or current computed value in rupees for operational reporting.[7]

- status changes from issued to overdue when due date is passed and no return exists, and to returned after return processing.[13][7]

Recommended uniqueness rule

The system must prevent the same user from holding the same book twice simultaneously. In PostgreSQL, this is best enforced with a partial unique index on active rows because partial unique indexes enforce uniqueness only on rows matching a condition.[8][14]

```
CREATE UNIQUE INDEX uq_active_book_per_user
ON issued_books (user_id, book_id)
WHERE status IN ('issued', 'overdue');
```

This allows historical returned records to exist while blocking duplicate active issues for the same user-book pair.[8][15]

6.4 Waitlist table

```
CREATE TABLE waitlist (
  id SERIAL PRIMARY KEY,
  user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  book_id INT NOT NULL REFERENCES books(id) ON DELETE CASCADE,
  position INT NOT NULL CHECK (position > 0),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Purpose

This table stores the waiting queue for books that currently have no available copies. The row with the lowest position has highest priority for the next allocation.[6][7]

Notes

- Queue order can be managed using position, but the backend must maintain it consistently after cancellation or fulfillment.[7]
- A student should not have multiple active waitlist entries for the same book, so a uniqueness rule is recommended.[7]

```
CREATE UNIQUE INDEX uq_waitlist_user_book
ON waitlist (user_id, book_id);
```

This version works because the table stores only active waitlist rows. Once an entry is fulfilled or cancelled, the row should be deleted or archived elsewhere.[7]

6.5 Book requests table

```
CREATE TABLE book_requests (
  id SERIAL PRIMARY KEY,
  user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title VARCHAR(255) NOT NULL,
  author VARCHAR(255) NOT NULL,
  status VARCHAR(20) NOT NULL CHECK (status IN ('pending', 'approved', '
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Purpose

This table stores student requests for books not currently available in the library catalog. It supports review and acquisition workflows by librarians.[6]

Notes

- A simple status lifecycle is enough for the current scope: `pending`, `approved`, `rejected`.[6]
- If the system later needs procurement tracking, extra fields such as `remarks`, `reviewed_by`, and `reviewed_at` can be added.[6]

7. Relationship model

7.1 Relationship overview

- One user can have many `issued_books` records.[6][4]
- One book can have many `issued_books` records over time.[6][4]
- One book can have many `waitlist` entries.[6][7]

- One user can have many `waitlist` entries, but only one per book at a time.[7]
- One user can create many `book_requests` records.[6]

7.2 Textual ER explanation

`users` is the parent identity table. `issued_books`, `waitlist`, and `book_requests` all depend on it through `user_id`.[3]

`books` is the parent catalog table. `issued_books` and `waitlist` both depend on it through `book_id`, allowing the system to track circulation history and demand against the same title record.[6][4]

Because there is no `book_copies` table in this version, all issue and waitlist logic is handled at title level. This simplifies the schema but makes the `books.available_copies` column operationally important and highly sensitive to correct transaction handling.[3][4]

8. Business rules mapped to schema

Business rule	Schema support	Enforcement method
A book can be issued for 7 days maximum	issued_books.issue_date, issued_books.due_date	Backend sets due date to issue date + 7 days.[6][13]
Fine is ₹5/day for late returns	issued_books.fine_amount	Backend computes fine from overdue days.[7]
If unavailable, student joins waitlist	waitlist table	Backend inserts queue entry.[6][7]
Returned book goes to next waitlisted student	waitlist plus transaction logic	Backend selects lowest queue position first.[7]
Prevent duplicate issue	Partial unique index on active issue rows	PostgreSQL unique partial index.[8][14]
Overdue loans must be identified	status, due_date, return_date	Backend or scheduled job updates status/view.[13]

9. Recommended indexes

The prompt already identifies three important indexes, and those should be included. PostgreSQL indexing improves search and relationship-heavy lookups when queries repeatedly filter by title, borrower, or waitlist target.[16][17]

```
CREATE INDEX idx_books_title ON books(title);
CREATE INDEX idx_issued_user ON issued_books(user_id);
CREATE INDEX idx_waitlist_book ON waitlist(book_id);
```

Additional recommended indexes

```
CREATE INDEX idx_issued_book ON issued_books(book_id);
CREATE INDEX idx_issued_status_due_date ON issued_books(status, due_da
CREATE INDEX idx_waitlist_book_position ON waitlist(book_id, position);
CREATE INDEX idx_book_requests_user ON book_requests(user_id);
```

These additional indexes help common librarian workflows such as checking overdue records, loading a book's waitlist in queue order, and viewing issue history by title.[16][17]

10. Transaction design

10.1 Issue book transaction

Issuing a book must be done inside a database transaction. The system should lock the selected book row, verify `available_copies > 0`, insert into `issued_books`, and decrement `available_copies` before commit.[10][1][11]

Suggested flow

1. Begin transaction.[11]
2. Lock target row in `books` for update.[10]
3. Check `available_copies > 0`.[6]
4. Check user does not already have an active issue for that book.[7][8]
5. Insert `issued_books` row with calculated due date.[6][13]
6. Update `books.available_copies = available_copies - 1`.[6]
7. Commit transaction.[11]

10.2 Return book transaction

Returning a book must also be transactional so fine, status, and stock updates stay consistent. If a waitlist exists, the transaction should allocate the returned availability to the next student rather than simply releasing general stock.[7][10]

Suggested flow

1. Begin transaction.[11]
2. Lock the open `issued_books` row and related `books` row.[10]
3. Set `return_date` and calculate final `fine_amount`.[7]
4. Mark status as `returned`.[13]
5. Check the earliest `waitlist` entry for the book.[7]
6. If waitlist exists, issue to next student or create the next circulation action immediately according to application flow; otherwise increment `available_copies`.[7]
7. Remove or resolve the waitlist entry if served.[7]
8. Commit transaction.[11]

11. Fine calculation logic

The schema stores the fine amount, but the actual formula belongs to service logic. The required rule is that fine equals ₹5 multiplied by the number of late days, and late days are counted only after the due date.[7]

Formula

$$extfine = \max(0, extcurrent_date - extdue_dateindays)imes$$

At return time, `current_date` becomes the actual return date. For overdue books not yet returned, the system may calculate a live value in queries or scheduled updates and also mark the row as `overdue`.[13][7]

12. Overdue handling strategy

The schema supports overdue tracking through `status`, `due_date`, and `return_date`. An issued row becomes overdue when the current timestamp is later than `due_date` and `return_date` is still null.[13]

This status can be handled in either of two practical ways:

- Update rows periodically through a scheduled job that marks records as `overdue`.[13]
- Derive overdue state dynamically in queries, while still storing the final result when the book is returned.[13][4]

For a small system, dynamic derivation plus an overdue listing query is often enough and avoids frequent batch updates.[4]

13. Validation and constraints

13.1 Database-level constraints

- Valid roles enforced by `CHECK` on `users.role`.[2][12]
- Non-negative inventory enforced by `CHECK` on `total_copies` and `available_copies`.[2][12]
- Available stock cannot exceed total stock through `chk_available_le_total`.[2][12]
- Fine amount must be non-negative.[2][12]
- Valid issue statuses enforced through `CHECK` on `issued_books.status`.[2][12]
- Date integrity enforced through `due_date >= issue_date` and `return_date >= issue_date`.[2][12]

13.2 Application-level validations

Some important rules are better handled in FastAPI service logic because they depend on current system state or multiple row checks:[1]

- Student cannot issue the same book twice simultaneously.[7]
- Student cannot join the waitlist if copies are available.[6][7]
- Student cannot join the same waitlist multiple times.[7]
- Returned stock should go to the first waitlisted user before being shown as generally available.[7]

14. Edge case handling

14.1 Multiple students waiting for the same book

The waitlist must always be processed by ascending `position`. The backend should avoid gaps or duplicate positions after an entry is removed or served.[7]

14.2 Same student trying to issue multiple copies of same book

This is blocked by business logic and reinforced by the partial unique index on active issues. Even if two requests arrive close together, PostgreSQL can help preserve the uniqueness rule for active rows.[8][14]

14.3 Fine accumulation over multiple days

Fine should not be calculated only once at return time conceptually. The system must be able to show a running fine for active overdue books based on current date minus due date.[13][7]

14.4 Overdue books not yet returned

These remain in `issued_books` with `return_date = NULL` and status either derived or updated to `overdue`. They should appear in overdue reports and librarian views until closed.[13][4]

14.5 Waitlist removal and reorder risk

If a middle waitlist entry is cancelled, the backend must shift later positions or switch to timestamp-based ordering consistently. Position management must never leave ambiguous priority order.[7]

15. Limitations of current schema

This version is intentionally simplified, but it has limits. Because there is no `book_copies` table, the system cannot identify exactly which physical copy was issued or returned, and it cannot represent copy-specific states such as one damaged copy among several owned copies.[6][4]

It also means automatic allocation from waitlist is done at title level rather than against a specific copy identity. For the current project size this is acceptable, but a later version should introduce copy-level tracking if the library wants better physical control or auditability.[6][4]

16. Recommended DDL script

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('librarian', 'student')),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE books (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    isbn VARCHAR(20) UNIQUE,
    total_copies INT NOT NULL CHECK (total_copies >= 0),
    available_copies INT NOT NULL CHECK (available_copies >= 0),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_available_le_total CHECK (available_copies <= total_co
);

CREATE TABLE issued_books (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    book_id INT NOT NULL REFERENCES books(id) ON DELETE CASCADE,
    issue_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    due_date TIMESTAMP NOT NULL,
    return_date TIMESTAMP,
    fine_amount INT NOT NULL DEFAULT 0 CHECK (fine_amount >= 0),
    status VARCHAR(20) NOT NULL CHECK (status IN ('issued', 'returned', 'ov
    CONSTRAINT chk_due_after_issue CHECK (due_date >= issue_date),
    CONSTRAINT chk_return_after_issue CHECK (return_date IS NULL OR re
);

CREATE TABLE waitlist (
    id SERIAL PRIMARY KEY,
```

```
    user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    book_id INT NOT NULL REFERENCES books(id) ON DELETE CASCADE,
    position INT NOT NULL CHECK (position > 0),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE book_requests (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    status VARCHAR(20) NOT NULL CHECK (status IN ('pending', 'approved', '
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX uq_active_book_per_user
ON issued_books (user_id, book_id)
WHERE status IN ('issued', 'overdue');

CREATE UNIQUE INDEX uq_waitlist_user_book
ON waitlist (user_id, book_id);

CREATE INDEX idx_books_title ON books(title);
CREATE INDEX idx_issued_user ON issued_books(user_id);
CREATE INDEX idx_waitlist_book ON waitlist(book_id);
CREATE INDEX idx_issued_book ON issued_books(book_id);
CREATE INDEX idx_issued_status_due_date ON issued_books(status, due_da
CREATE INDEX idx_waitlist_book_position ON waitlist(book_id, position);
CREATE INDEX idx_book_requests_user ON book_requests(user_id);
```

17. Implementation recommendation

This schema is suitable for the current scope and follows the simplified design you asked for. The most important implementation requirement is not just table creation, but correct transaction handling in FastAPI service code during issue and return operations so counters, fine values, and waitlist actions remain consistent.[10][1][11]

For the viva or documentation defense, the strongest explanation is that the schema is intentionally simple for a 200-book system, but still uses PostgreSQL strengths such as constraints, foreign keys, indexes, and partial unique indexes to avoid the common mistakes of duplicate issues, broken inventory counts, and unordered waitlist handling.[8][16][17]